Guia de estudo: formulários e responsividade

Interface Web com HTML e CSS: layout com Grid, formulários, pseudo-seletores e media queries, com exemplos do projeto SATEC 2026

Material para estudar em casa. Comece pelo **roteiro da seção 1** e use as outras seções como consulta enquanto revisa o seu `estilo.css`. Cada seção tem um link para o **MDN**, a referência oficial.

1. Roteiro de estudo

O quê	Onde	Foco da leitura ou do exercício
1. Formulários acessíveis	MDN: Formulários web	Ler “Seu primeiro formulário” e “Como estruturar um formulário web”. Prestar atenção em `<label>`, `<fieldset>` e `<legend>`.
2. Media queries	MDN: Usando media queries	Sintaxe de `@media`, `min-width` e `max-width`. Pular a parte de `and`, `not` e `orientation` se o tempo apertar.
3. Grid Garden	cssgridgarden.com/#pt-br	Jogo em português. Os primeiros níveis treinam `grid-column-start`, `grid-column-end` e `grid-column`, inclusive com números negativos e `span`.
4. Aplicar na SATEC (≈15 min)	Seu projeto + DevTools	Conferir a página em 375px e em 1280px (seção 11) e salvar os dois prints no portfólio.
5. Para ir além (opcional)	web.dev: Learn Responsive Design	Curso gratuito do Google sobre design responsivo moderno, incluindo container queries (seção 9). Em inglês, com tradução automática do navegador.

2. Layout da página com áreas ver no MDN ↗️

O `grid-template-areas` desenha a página com nomes. Cada filho direto do `<body>` recebe o nome da sua área com `grid-area`.

```
body {
  display: grid;
  grid-template-columns: 1fr 280px;   /* conteúdo flexível | lateral fixa */
  grid-template-areas:
    "pular    pular"
    "topo     topo"
    "conteudo lateral"
    "rodape   rodape";
}

header { grid-area: topo; }
main   { grid-area: conteudo; }
aside  { grid-area: lateral; }
footer { grid-area: rodape; }
```

Cada linha entre aspas é uma linha da grade; um nome repetido faz a área ocupar mais de uma coluna. Os nomes no mapa e no `grid-area` precisam ser **idênticos**.

3. grid-column e as linhas numeradas ver no MDN ↗️

Uma grade de 2 colunas tem **3 linhas verticais**, numeradas da esquerda para a direita (1, 2, 3) e também de trás para frente (-1 é sempre a última).

Escrita	Significado	Exemplo (SATEC 2026)
`grid-column: 2;`	Item fica só na coluna 2	`form button { grid-column: 2; }`
`grid-column: 1 / 3;`	Da linha 1 até a linha 3 (duas colunas)	`footer > p { grid-column: 1 / 3; }`

Escrita	Significado	Exemplo (SATEC 2026)
`grid-column: 1 / -1;`	Da primeira à última linha: ocupa a largura toda, com qualquer número de colunas	`footer > a { grid-column: 1 / -1; }`
`grid-column: span 2;`	Ocupa 2 colunas a partir de onde está	`footer section { grid-column: span 2; }`
`justify-self: start;`	Alinha só aquele item dentro da célula	`form button { justify-self: start; }`

4. Formulário bem estruturado (HTML) ver no MDN ↗️

```
<form action="inscricao.html" method="post">
  <fieldset>
    <legend>Dados pessoais</legend>
    <div class="campo">
      <label for="email">E-mail</label>
      <input type="email" id="email" name="email"
             autocomplete="email" aria-describedby="dica-email" required>
      <p id="dica-email" class="dica">A confirmação será enviada para este e-mail.</p>
    </div>
  </fieldset>
  <button type="submit">Enviar inscrição</button>
</form>
```

Prática	Por quê	Na SATEC
`<label for="id">`	Leitor de tela anuncia o nome do campo; clicar no texto foca o campo	Todos os campos
`placeholder`	**Não** substitui o label: some quando o usuário digita e costuma ter pouco contraste	Não usamos
`type="email"`	Teclado com “@” no celular e validação de formato sem JavaScript	Campo de e-mail
`autocomplete`	O navegador preenche nome e e-mail sozinho	`name`, `email`
`required`	Validação nativa: o navegador impede o envio do campo vazio	Todos os campos
`<fieldset>` + `<legend>`	Agrupa campos relacionados; o leitor de tela anuncia o grupo	Dados pessoais, Dados acadêmicos
`aria-describedby`	Liga uma dica ao campo; ela é lida junto com o nome	Dica do e-mail

A `div.campo` só junta cada label ao seu campo para o layout. É o mesmo uso correto de `<div>` da `div.cards`: um agrupamento sem significado.

5. Tipos de campo do HTML5 ver no MDN ↗️

O atributo `type` do `<input>` muda o **teclado no celular**, o **controle** que o navegador desenha e a **validação** automática. Escolher o tipo certo é uma boa prática que não custa nenhuma linha de CSS ou JavaScript.

Tipo	O que o navegador oferece	Uso possível na SATEC
`text`	Campo de texto livre (é o padrão quando não há type)	Nome, curso
`email`	Teclado com “@”; valida o formato nome@dominio	E-mail da inscrição
`tel`	Teclado numérico de telefone (não valida o formato sozinho)	WhatsApp para avisos
`url`	Teclado com “/” e “.com”; exige http:// ou https://	Link do GitHub ou portfólio

Tipo	O que o navegador oferece	Uso possível na SATEC
`number`	Setas e teclado numérico; aceita min, max e step	Quantidade de acompanhantes
`range`	Controle deslizante entre min e max	Nota de avaliação do evento
`date / time`	Calendário e relógio nativos	Data da oficina escolhida
`datetime-local`	Data e hora juntas, sem fuso horário	Agendar visita ao estande
`month / week`	Escolha de mês ou semana (suporte varia entre navegadores)	pouco usados
`color`	Seletor de cor	pouco usado em inscrições
`search`	Campo de busca, com botão para limpar em alguns navegadores	Buscar uma palestra
`password`	Esconde o que é digitado	Área do participante
`checkbox / radio`	Marcar vários itens / escolher um só do grupo (mesmo name)	Aceite dos termos / turno
`file`	Escolha de arquivo; accept limita os tipos	Enviar foto para o crachá

Atributo	Para que serve	Exemplo
`required`	Campo obrigatório	`<input type="text" required>`
`min / max / step`	Limites e passo de number, range e date	`<input type="number" min="0" max="2">`
`minlength / maxlength`	Tamanho mínimo e máximo do texto	`<input type="text" minlength="3">`
`pattern`	Formato exigido, escrito como expressão regular	`pattern="[0-9]{11}"`
`inputmode`	Só troca o teclado do celular, sem validar	`<input type="text" inputmode="numeric">`
`autocomplete`	Diz ao navegador o que preencher	`autocomplete="tel"`

Cuidado: a validação do navegador ajuda o usuário, mas **não** protege o servidor. Quem recebe os dados (o back-end) sempre precisa validar de novo.

6. Estilizar campos (CSS) ver no MDN ↗️

Propriedade	Por quê	Exemplo (SATEC 2026)
`font: inherit;`	input, select e button não herdam a fonte do body sozinhos	`input, select, button { font: inherit; }`
`width: 100%;`	Campo ocupa a largura da sua coluna	`input[type="email"] { width: 100%; }`
`min-height: 2.75rem;`	44px: área de toque confortável para o dedo	`select { min-height: 2.75rem; }`
`border`	Contorno visível: contraste mínimo de 3:1 com o fundo	`input { border: 1px solid #64748b; }`
`accent-color`	Muda a cor do checkbox marcado sem recriar o controle	`.aceite input { accent-color: #1e3a8a; }`
`display: grid;`	Label em cima, campo embaixo: funciona em qualquer largura	`.campo { display: grid; gap: 0.375rem; }`

O seletor de atributo `input[type="email"]` escolhe só aquele tipo de campo. Sem ele, `width: 100%` também esticaria o checkbox.

7. Pseudo-classes e pseudo-elementos ver no MDN ↗️

São seletores que o HTML não escreve: o navegador sabe sozinho em que **estado** ou **posição** o elemento está. A **pseudo-classe** usa um dois-pontos (`:hover`) e escolhe elementos num estado; o **pseudo-elemento** usa dois (`::before`) e cria ou escolhe uma **parte** de um elemento.

Pseudo-classe	Quando vale	Exemplo (SATEC 2026)
`:hover`	Mouse sobre o elemento (no celular, quase não existe)	`.botao:hover { background-color: #1e40af; }`
`:focus`	Elemento com foco, por teclado ou por clique	`prefira :focus-visible`
`:focus-visible`	Foco vindo do teclado (Tab). Mostra onde o usuário está	`input:focus-visible { outline: 3px solid #0d9488; }`
`:invalid / :valid`	Campo inválido ou válido desde o carregamento da página	`evite para mostrar erros`
`:user-invalid`	Inválido só depois que o usuário interagiu ou tentou enviar	`input:user-invalid { border-color: #b91c1c; }`
`:required`	Campos com o atributo required	`input:required { border-left-width: 4px; }`
`:checked`	Checkbox ou radio marcado	`#termos:checked + label { font-weight: bold; }`
`:disabled`	Campo ou botão desabilitado	`button:disabled { opacity: 0.5; }`
`:first-child / :last-child`	Primeiro ou último filho do pai	`nav li:last-child { margin-left: auto; }`
`:nth-child(even)`	Filhos em posição par (odd = ímpar, 3n = de 3 em 3)	`tbody tr:nth-child(even) { background: #f1f5f9; }`
`:not(...)`	Todos, menos os que combinam com o seletor	`input:not([type="checkbox"]) { width: 100%; }`
`:has(...)`	Elemento que contém algo (“seletor de pai”)	`.campo:has(:user-invalid) label { color: #b91c1c; }`
Pseudo-elemento	**O que escolhe**	**Exemplo (SATEC 2026)**
`::before / ::after`	Conteúdo decorativo antes ou depois do elemento (precisa de content)	`h2::before { content: "# "; color: #0d9488; }`
`::placeholder`	Texto de exemplo dentro do campo	`input::placeholder { color: #64748b; }`
`::marker`	Marcador de item de lista (bolinha ou número)	`li::marker { color: #0d9488; }`
`::selection`	Trecho que o usuário selecionou com o mouse	`::selection { background: #ccfbf1; }`
`::first-letter`	Primeira letra de um bloco de texto	`#titulo-principal::first-letter { color: #0d9488; }`

Regra de ouro: nunca escreva `outline: none` sem colocar outro indicador de foco no lugar. E a cor não deve ser o único sinal de erro: a mensagem do navegador (“Preencha este campo”) continua aparecendo. Texto colocado com `::before` e `::after` é decorativo: informação importante fica no HTML.

8. Media queries ver no MDN ↗️

```
/* Sem media query: vale para todas as telas (celular primeiro) */
h1 { font-size: 2rem; }

/* Só quando a área visível tem 900px ou mais */
@media (min-width: 900px) {
  h1 { font-size: 2.5rem; }
}
```

Escrita	Significado	Estratégia
`@media (min-width: 600px)`	“A partir de” 600px de largura	**Mobile first**: começa no celular e acrescenta regras
`@media (max-width: 599px)`	“Até” 599px de largura	Desktop first: começa no desktop e desfaz regras
`<meta name="viewport" …>`	Sem ela, o celular finge ter 980px e as media queries não funcionam como esperado	Já está no `<head>` da SATEC

A media query **não** aumenta a especificidade. Se a regra base vier **depois** do `@media`, ela vence. Escreva as media queries no fim do arquivo.

9. Mobile first: na SATEC e em 2026 ver no MDN ↗️

Na SATEC, a regra base descreve o celular e cada media query acrescenta o que muda numa tela maior:

Largura	O que muda	Regras
Base (celular)	Uma coluna: topo, conteúdo, lateral e rodapé empilhados; título menor; menu em botões que preenchem a linha	`grid-template-areas: "topo" "conteudo" "lateral" "rodape";`
600px ou mais	Dois campos por linha no formulário; patrocinadores em 3 colunas; menu numa linha só; rodapé em 2 colunas	`fieldset { grid-template-columns: repeat(2, 1fr); }`
900px ou mais	O `<aside>` sobe para a lateral; patrocinadores voltam a 2 colunas; logo, menu e botão lado a lado no cabeçalho	`body { grid-template-columns: 1fr 280px; }`

O breakpoint é escolhido **onde o conteúdo quebra**, não pelo modelo do aparelho. Os cards dos palestrantes nem precisaram de media query: `repeat(auto-fit, minmax(220px, 1fr))` já se adapta sozinho.

Mobile first em 2026. A técnica continua sendo o ponto de partida recomendado, por três motivos:

- **O público está no celular:** mais da metade dos acessos à web no mundo vem de celulares, segundo medições como a do StatCounter. No Brasil, para muita gente o celular é o único acesso à internet.
- **Os buscadores olham a versão móvel:** o Google indexa e avalia os sites pela versão que aparece no celular (mobile-first indexing). Conteúdo escondido no celular pode “sumir” da busca.
- **Menos código para desfazer:** começar pelo layout de uma coluna e acrescentar regras é mais simples do que montar o desktop e depois desmontar.

O que mudou é que, hoje, **media query não é a única ferramenta**. O CSS moderno tem recursos que todos os navegadores atuais suportam e que reduzem a quantidade de breakpoints:

Recurso	O que resolve	Exemplo
Layout intrínseco	A grade decide sozinha quantas colunas cabem (já usamos nos cards)	`repeat(auto-fit, minmax(220px, 1fr))`
Container queries	O componente se adapta ao espaço do **pai**, não ao tamanho da tela. Ideal para cards que aparecem em lugares de larguras diferentes	`@container (min-width: 450px) { ... }`
clamp()	Tamanho fluido com mínimo e máximo, sem degraus de media query	`h1 { font-size: clamp(2rem, 5vw, 2.5rem); }`
Preferências do usuário	Respeita o modo escuro e quem pede menos animação no sistema	`@media (prefers-color-scheme: dark)`

Recurso	O que resolve	Exemplo
Tipo de ponteiro	Ajusta para toque ou mouse, em vez de "adivinhar" pela largura	`@media (pointer: coarse) { ... }`
Unidades dvh / svh	Altura real da tela do celular, descontando a barra do navegador	`min-height: 100dvh;`

```
/* Container query: os patrocinadores olham o tamanho da seção, não da tela */
aside section { container-type: inline-size; }

@container (min-width: 450px) {
  aside ul { grid-template-columns: repeat(3, 1fr); }
}
```

Lembra dos patrocinadores que iam de 2 para 3 e voltavam para 2 colunas? Com a container query, uma regra só resolve: a lista ganha 3 colunas sempre que **a seção onde ela está** tiver 450px ou mais, seja no celular deitado, seja no `<aside>` estreito do desktop.

Resumo para 2026: comece pelo celular, deixe o Grid e o Flexbox se adaptarem sozinhos sempre que possível, use **media queries** para mudar a estrutura da **página** e **container queries** para mudar um **componente**.

10. Cabeçalho e menu responsivos

```
header {
  display: grid;
  grid-template-columns: 1fr auto;
  grid-template-areas:
    "logo   botao"
    "slogan slogan"
    "menu   menu";
}

@media (min-width: 900px) {
  header {
    grid-template-columns: auto 1fr auto;
    grid-template-areas:
      "logo slogan botao"
      "logo menu   botao";
  }
}
```

Grid fora, Flexbox dentro: o `header` usa Grid porque cada parte tem um lugar fixo; o `ul` do menu continua em Flexbox porque os links são uma fileira só. No celular, `header nav li { flex-grow: 1; }` faz cada linha do menu ficar cheia.

11. Testar no DevTools

- **Modo dispositivo:** F12 e depois **Ctrl+Shift+M**. Escolha uma largura (375px é um celular comum) ou arraste a borda da página.
- **Onde quebra?** Arraste a largura devagar e observe o momento em que o layout muda. É ali que está cada `@media`.
- **Rolagem horizontal:** se aparecer uma barra embaixo no celular, algum elemento é mais largo que a tela. Procure larguras fixas.
- **Selo grid:** no painel Elements, clique no selo **grid** ao lado do `<body>` para ver as linhas e os nomes das áreas.
- **Só teclado:** navegue pelo formulário apenas com Tab e Shift+Tab. Se em algum momento você não souber onde está o foco, falta `:focus-visible`.

12. Erros comuns

- **Media query antes da regra base:** a regra que vem depois vence. Coloque os `@media` no fim do arquivo.
- **Nome de área diferente:** `grid-area: lado` não encontra a área `lateral`. O item cai numa linha extra no fim da grade.
- **`outline: none`:** some o indicador de foco e quem usa teclado se perde. Use `:focus-visible` com um contorno visível.
- **`:invalid` no lugar de `:user-invalid`:** o formulário fica vermelho antes de o usuário digitar.

- **Placeholder como label:** o texto some ao digitar. Todo campo precisa de `<label for>`.
- **Fonte diferente nos campos:** faltou `font: inherit` em `input`, `select` e `button`.
- **Pseudo-elemento sem content:** `::before` e `::after` só aparecem com a propriedade `content` (mesmo que vazia: `content: ""`).
- **Campo com o tipo errado:** `type="text"` para e-mail ou telefone tira do usuário o teclado certo e a validação automática.
- **`nav a` em vez de `header nav a`:** o estilo do menu principal também muda o menu “Dia 1 | Dia 2 | Dia 3”.

13. Onde consultar e praticar

- **MDN Web Docs (oficial, em português):** developer.mozilla.org/pt-BR. Formulários: “Formulários web”; tipos de campo: elemento `<input>`; seletores: “Pseudo-classes” e “Pseudo-elementos”; layout: “Design responsivo” e “Container queries”.
- **web.dev (Google):** cursos gratuitos Learn Responsive Design e Learn Forms, atualizados com as técnicas atuais (em inglês).
- **Can I use:** caniuse.com mostra quais navegadores suportam cada recurso, como `:has()`, `:user-invalid` ou container queries.
- **Guia WCAG (em português):** guia-wcag.com explica as regras de acessibilidade (foco visível, contraste, rótulos de formulário) com exemplos.
- **Validador do W3C:** validator.w3.org confere o HTML, incluindo `label for` sem `id` correspondente.
- **Jogos:** Grid Garden (cssgridgarden.com/#pt-br) para `grid-column` e Flexbox Froggy (flexboxfroggy.com/#pt-br) para revisar Flexbox, ambos em português.
- **DevTools:** modo dispositivo (Ctrl+Shift+M), selos **grid** e **flex** no painel Elements e o painel **Accessibility**, que mostra o nome que o leitor de tela lê em cada campo.

14. IA no desenvolvimento web em 2026

Em 2026, assistentes de IA fazem parte do dia a dia de quem desenvolve para a web: estão dentro do editor de código, completam linhas enquanto você digita, explicam erros e conseguem gerar uma página inteira a partir de uma descrição. Isso **não** diminui a importância do que vimos na disciplina. Muda o papel de quem programa: menos digitação, mais **decisão e revisão**.

Onde a IA ajuda bem:

- **Explicar:** “o que faz `grid-column: 1 / -1`?”, “por que meu `::before` não aparece?”. Peça a explicação linha a linha, não só o código pronto.
- **Rascunhar:** gerar uma primeira versão de um formulário ou de uma media query que você depois revisa e ajusta.
- **Revisar:** pedir uma checagem de acessibilidade (“falta label em algum campo?”, “o contraste é suficiente?”) e conferir cada apontamento.

Onde a IA costuma errar, e você precisa perceber:

- **Más práticas que “funcionam”:** `outline: none`, `placeholder` no lugar de `<label>`, `<div>` clicável no lugar de `<button>`, larguras fixas em px que quebram no celular.
- **Código a mais:** frameworks, bibliotecas ou JavaScript para algo que o HTML e o CSS resolvem sozinhos (validação com `required`, menu sem hambúrguer, layout com Grid).
- **Recursos desatualizados ou sem suporte:** confira no MDN e no Can I use antes de usar.
- **Ela não vê a sua página:** a IA não testa em 375px, não navega com Tab e não ouve o leitor de tela. Quem testa é você.

Como usar nesta disciplina: entenda antes de colar (se não consegue explicar uma linha, ela não entra no projeto), teste em pelo menos duas larguras e com o teclado, e confira as propriedades novas neste guia ou no MDN. Em atividades avaliadas, siga a orientação do professor sobre o uso de IA e registre onde ela ajudou.